GPU-Parallel Lid-Driven Cavity Solver

Vorticity–Stream Function Formulation of the 2-D Navier–Stokes Equations in CUDA C

Course Project 2 – OE5450 Numerical Techniques in Ocean Hydrodynamics (Tutorial 6, Jan–May 2025)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras

Revised report, October 2026

Abstract

The steady flow in a lid-driven square cavity at Re = 100 is computed with an explicit vorticity–stream function finite-difference method whose four stencil operations (Thom wall vorticity, vorticity transport, red–black Gauss–Seidel stream-function relaxation and velocity recovery) are CUDA kernels with one thread per grid node. The submitted code could not produce a result: its time step was 6.4 times above the explicit diffusion limit derived in the original handwritten notes ($\Delta t \le h^2\mathrm{Re}/4$), so the vorticity overflowed within a dozen steps; it also wrote one 11 MB file per time step, and the output and velocity routines indexed the arrays transposed. With these errors corrected (and the steady-state tolerance scaled with the time step), the unmodified kernels were verified on three grids (129^2, 257^2, 401^2) by running them serially on the CPU through a CUDA emulation layer. The primary vortex ($\psi_{min} = -0.10350$ at $(0.616, 0.737)$) and the centreline velocities agree with Ghia et al. (1982) within the accuracy of that benchmark; a three-grid study gives an observed order of accuracy of 2.01–2.02 for ψ_{min}, u_{min}, v_{max} and v_{min}, Richardson-extrapolated values $\psi_{min} = -0.10352$, $u_{min} = -0.21405$, $v_{max} = 0.17957$, $v_{min} = -0.25380$, and a grid convergence index below 0.05% on the finest grid. `run.sh` repeats all runs on an NVIDIA GPU when one is present and records the GPU wall times next to the single-core baseline reported here.

Contents

1 Problem definition **3**

2 Numerical method **3**

2.1 Discretisation . . . 3

2.2 Coupling and pseudo-transient convergence . . . 3

2.3 Stability . . . 4

2.4 CUDA implementation . . . 4

2.5 Verification without a GPU: CPU emulation of the kernels . . . 5

3 Code review of the submitted program **5**

4 Results **6**

4.1 Convergence history . . . 6

4.2 Flow field . . . 7

4.3 Comparison with Ghia, Ghia & Shin (1982) . . . 7

4.4 Grid convergence study 8
4.5 Run times 9

5 Conclusions **9**

A Source code **10**

1 Problem definition

A unit square cavity, $0 \le x, y \le 1$, is filled with an incompressible Newtonian fluid; the lid $y = 1$ moves with $U = 1$ and the other walls are at rest. In two dimensions the Navier–Stokes equations can be written for the vorticity $\omega = \partial v/\partial x - \partial u/\partial y$ and the stream function ψ ($u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$), which satisfies continuity identically and removes the pressure:

$$\frac{\partial \omega}{\partial t} + u\frac{\partial \omega}{\partial x} + v\frac{\partial \omega}{\partial y} = \frac{1}{\mathrm{Re}}\nabla^2\omega, \tag{1}$$

$$\nabla^2\psi = -\omega, \tag{2}$$

with $\mathrm{Re} = UL/\nu = 100$, $\psi = 0$ on all walls (the walls form one streamline) and the no-slip condition $\partial\psi/\partial n = 0$ (moving lid: $\partial\psi/\partial y = U$). The flow starts from rest; the steady state is the long-time limit.

2 Numerical method

2.1 Discretisation

On a uniform grid $x_i = ih$, $y_j = jh$, $i, j = 0 \ldots N-1$, $h = 1/(N-1)$, Eq. (1) is advanced with forward Euler and second-order central differences (FTCS),

$$\omega_{ij}^{n+1} = \omega_{ij}^{n} + \Delta t\Big[\frac{\omega_{i+1,j}^{n} + \omega_{i-1,j}^{n} + \omega_{i,j+1}^{n} + \omega_{i,j-1}^{n} - 4\omega_{ij}^{n}}{\mathrm{Re}\,h^2} - \frac{\psi_{i,j+1} - \psi_{i,j-1}}{2h}\frac{\omega_{i+1,j}^{n} - \omega_{i-1,j}^{n}}{2h} + \frac{\psi_{i+1,j} - \psi_{i-1,j}}{2h}\frac{\omega_{i,j+1}^{n} - \omega_{i,j-1}^{n}}{2h}\Big], \tag{3}$$

and Eq. (2) by the five-point Laplacian. The wall vorticity follows from a Taylor expansion of ψ normal to the wall with $\psi_w = 0$ and $\partial\psi/\partial n = U_w$ (Thom's formula, first-order accurate at the wall but globally second-order):

$$\omega_{lid} = -\frac{2\psi_{i,N-2}}{h^2} - \frac{2U}{h}, \qquad \omega_{bottom} = -\frac{2\psi_{i,1}}{h^2}, \qquad \omega_{left} = -\frac{2\psi_{1,j}}{h^2}, \qquad \omega_{right} = -\frac{2\psi_{N-2,j}}{h^2}. \tag{4}$$

The velocities are recovered at the end, $u = (\psi_{i,j+1} - \psi_{i,j-1})/2h$, $v = -(\psi_{i+1,j} - \psi_{i-1,j})/2h$, with $u = U$ on the lid.

2.2 Coupling and pseudo-transient convergence

Each time step performs *one* red–black Gauss–Seidel sweep of Eq. (2) (the algorithm of the original notes). The stream function is therefore not solved exactly at every step but relaxed in pseudo-time together with ω; the fixed point of the coupled iteration is the exact steady solution of the discrete equations. For Gauss–Seidel the spectral radius of the slowest mode is $\rho_{GS} = \cos^2(\pi h) \approx 1 - \pi^2 h^2$, so the stream function follows the vorticity with a lag time

$$\tau_\psi \approx \frac{\Delta t}{1 - \rho_{GS}} \approx \frac{\Delta t}{\pi^2 h^2} = 1.66,\ 2.32,\ 2.43 \quad \text{for } N = 129,\ 257,\ 401, \tag{5}$$

comparable with the viscous decay time of the flow. This explains (i) why the steady state needs $t \approx 11$–25 time units, and (ii) why the stopping test must be stated in terms of $\partial\omega/\partial t$ rather than the per-step change (Section 3).

2.3 Stability

Von Neumann analysis of the FTCS transport equation (original notes, after Juujärvi & Kinnunen) gives the diffusion limit and the cell-Reynolds-number / advective limits

$$d = \frac{\Delta t}{\mathrm{Re}\, h^2} \le \frac{1}{4} \;\Leftrightarrow\; \Delta t \le \frac{h^2 \mathrm{Re}}{4}, \qquad \mathrm{Re}_h = U h\, \mathrm{Re} \le 2, \qquad \Delta t \le \frac{2}{\mathrm{Re}\, U^2}. \tag{6}$$

Grid	h	Δt	d	Re_h	steps to steady state (default / tight)
129^2	1/128	1.0×10^{-3}	0.164	0.78	10 940 / 19 586
257^2	1/256	3.5×10^{-4}	0.229	0.39	39 616 / 70 655
401^2	1/400	1.5×10^{-4}	0.240	0.25	95 127 / 169 301
original	1/400	1.0×10^{-3}	**1.600**	0.25	diverges (overflow at step ≈ 12)

Table 1: Time steps and stability numbers. "Default": stop when $\max|\omega^{n+1} - \omega^n|/\Delta t < 10^{-2}$ (the code); "tight": $< 10^{-4}$ (grid study).

2.4 CUDA implementation

The fields $\omega, \omega^{old}, \psi, u, v$ ($5 \times 8N^2$ bytes, 6.4 MB at $N = 401$) are allocated in unified memory; arrays are stored as `[i*NY+j]`. A 16×16 thread block covers a 16×16 patch of nodes, and the launch grid is $\lceil N/16 \rceil^2$ blocks; each thread updates one node, so every kernel is a pure stencil with no shared state:

- `apply_boundary_conditions`: Thom vorticity on the four walls (boundary threads only);
- `update_vorticity`: FTCS update of interior nodes reading the previous field `Vop` (a device-to-device copy made before the kernel), so no thread reads a value written in the same launch;
- `solve_stream_function`: launched twice per step with `phase`$= 0, 1$; a thread acts only if $(i + j) \bmod 2 =$`phase`. Red nodes depend only on black neighbours and vice versa, so each half-sweep is race-free and the result is identical to the serial red–black Gauss–Seidel sweep;
- `calculate_velocity`: central differences of ψ (output steps only).

The convergence test (maximum of $|\omega - \omega^{old}|$) is a host loop over the unified-memory arrays. It is correct but, on a GPU, migrates the arrays to the host every step; a device reduction (e.g. `thrust::transform_reduce`) evaluated every 100 steps would remove this traffic. It was left unchanged because it is not an error.

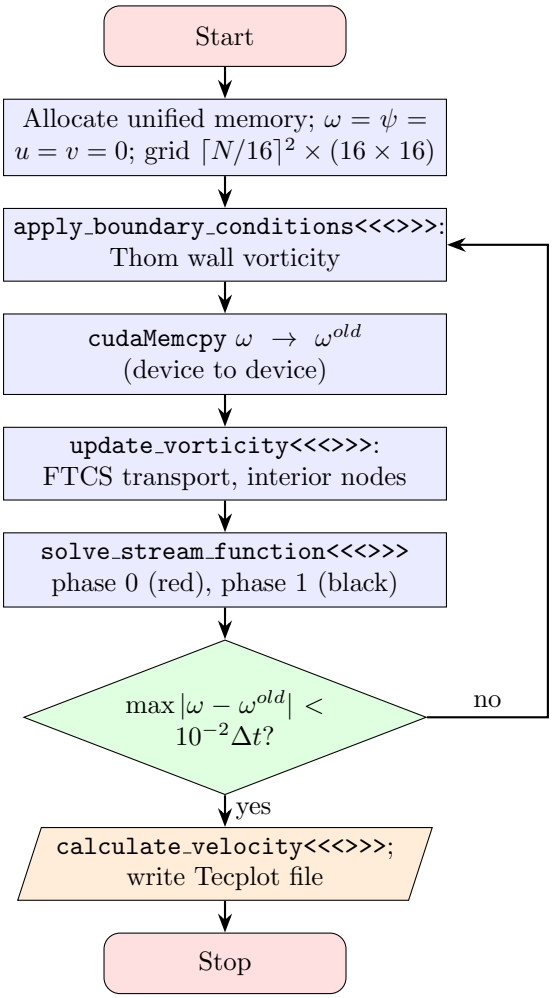


Figure 1: Time loop of `code/Lid_driven_Cavity.cu`; every box marked `<<<>>>` is a CUDA kernel with one thread per grid node.

2.5 Verification without a GPU: CPU emulation of the kernels

The workspace used for this revision has no NVIDIA GPU, so the kernels were executed by a CUDA emulation layer (`tools/cuda_cpu_shim.h`, `tools/emulate.sh`). The script rewrites each launch `k<<<G,B>>>(args)` into a loop over `blockIdx` and `threadIdx` that calls the *unmodified* kernel body once per thread, and maps `cudaMallocManaged`, `cudaMemcpy` and `cudaDeviceSynchronize` to their host equivalents. Because every kernel here writes disjoint data and reads only data not written in the same launch (the red–black split guarantees this for the stream-function sweep), sequential execution gives the same result as the GPU apart from floating-point contraction. The CPU times quoted below are therefore single-core baselines; `run.sh` builds the same source with `nvcc -O3 -arch=native` and runs it on the GPU when one is available (results in `results/run_<N>_gpu` and `results/timing.txt`).

3 Code review of the submitted program

The submitted file is kept as `original/Lid_driven_Cavity_original.cu`; all changes are marked `[FIX]`, `[ADDED]` or `[CHANGED]` in `code/Lid_driven_Cavity.cu`. The numerical scheme and kernel structure were not altered.

Item	Error and correction
`#define Dt 0.001`	With $h = 0.0025$ the diffusion number is $d = 1.6$, 6.4 times the limit $\Delta t \leq h^2\mathrm{Re}/4 = 1.5625 \times 10^{-4}$ derived in the original notes; the emulated run overflows to `inf` within about 12 steps. Set to 1.5×10^{-4} ($d = 0.24$).
`#define Error 1e-5`	The test is on $\max\lvert\omega^{n+1} - \omega^n\rvert = \Delta t\,\lvert\partial\omega/\partial t\rvert$; with the stable Δt it stopped the 401^2 run at $t = 8.7$ while ψ still lagged ($\max\lvert\nabla^2\psi + \omega\rvert = 0.11$, ψ_{min} 1.8% off). Replaced by $10^{-2}\Delta t$, i.e. $\lvert\partial\omega/\partial t\rvert < 10^{-2}$, identical to the original value at $\Delta t = 10^{-3}$.
`MaxIter 50000`	$t_{max} = 7.5$ at the stable time step, shorter than the transient; raised to 400 000 (the convergence test ends the run).
`OUTPUT_INTERVAL 1`	One 11 MB Tecplot file per step, about 550 GB for 50 000 steps; set to 20 000.
`write_tecplot2D`	Index `j*nx+i` while the solver stores `i*NY+j`: every field was written transposed. Fixed.
`calculate_velocity`	Index `j*NX+i` (differentiates ψ along the wrong direction) and an early `return` for all boundary nodes that made the lid-velocity assignment unreachable. Fixed: consistent indexing, wall and lid values set explicitly.
final output	Final file named after `MaxIter`; now `flow_999999.dat`.
additions	Wall-clock timing (`clock_gettime`), convergence history file, progress print every 1000 steps.

Table 2: Errors found in the submitted code and their corrections.

4 Results

4.1 Convergence history

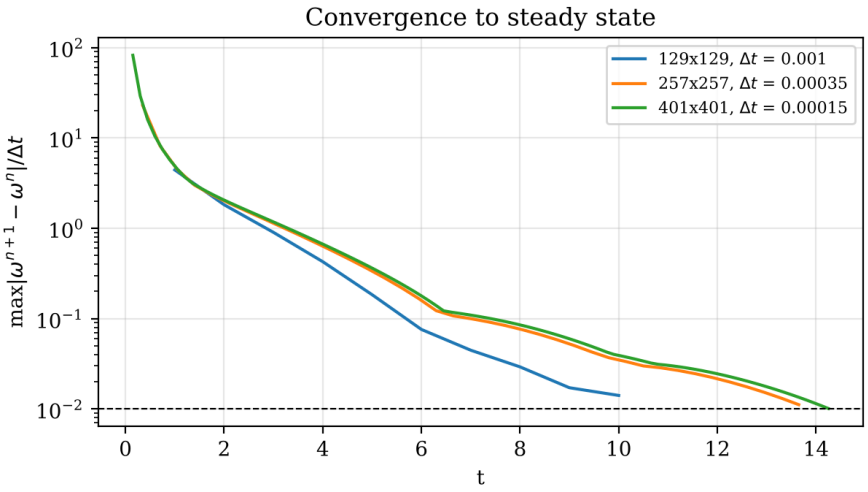


Figure 2: Rate of change of the vorticity, $\max\lvert\omega^{n+1} - \omega^n\rvert/\Delta t$, for the three grids (default stopping tolerance, dashed).

After the start-up transient the residual decays exponentially. The finer grids decay more slowly because the stream-function lag τ_ψ grows with $\Delta t/h^2$, which is nearly constant here ($d \approx 0.16$–0.24), so the number of steps grows roughly as h^{-2}.

4.2 Flow field

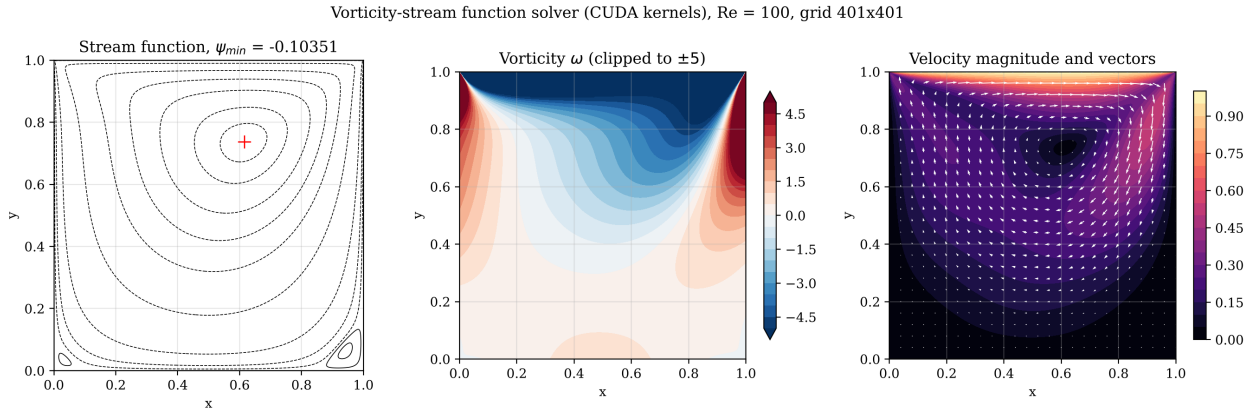


Figure 3: Stream function (vortex centre marked; contour levels down to 10^{-5} show both corner eddies), vorticity and velocity on the 401^2 grid.

The primary clockwise vortex sits above and downstream of the cavity centre. The vorticity is generated in the lid boundary layer and is singular at the two lid corners, where the velocity jumps from 1 to 0; vorticity of the opposite sign is produced on the downstream wall where the flow turns down. Two counter-rotating eddies occupy the bottom corners, four to five orders of magnitude weaker than the primary vortex.

4.3 Comparison with Ghia, Ghia & Shin (1982)

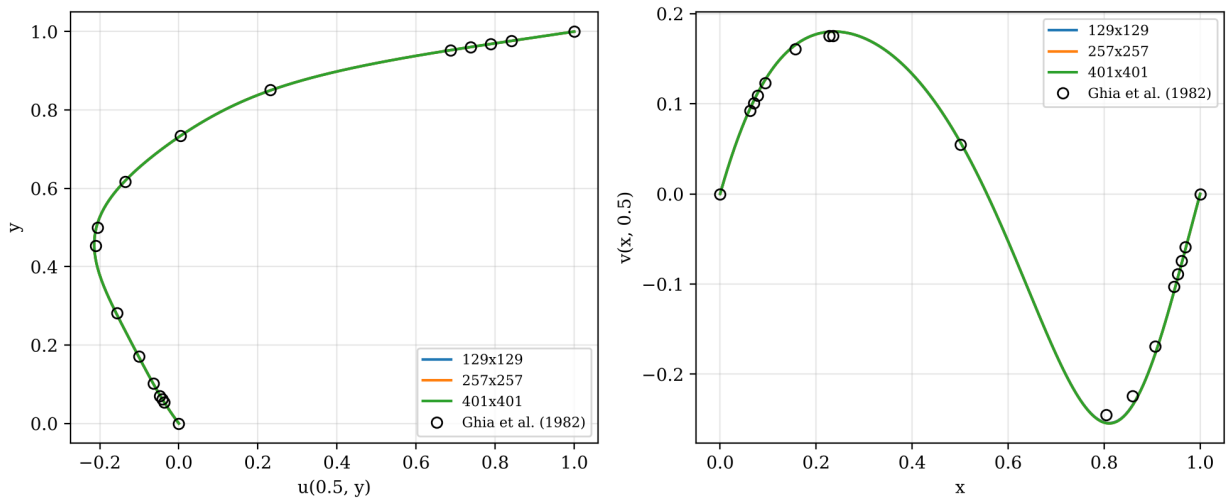


Figure 4: Centreline velocities $u(0.5, y)$ and $v(x, 0.5)$ against the benchmark.

Quantity	129^2	257^2	401^2	Ghia et al.
ψ_{min}	-0.103335	-0.103476	-0.103503	-0.103423
centre (x, y) (default runs)	(0.616, 0.737)	(0.616, 0.737)	(0.616, 0.737)	(0.617, 0.734)
u_{min} on $x = 0.5$	-0.213444	-0.213896	-0.213985	-0.21090
v_{max} on $y = 0.5$	0.178882	0.179403	0.179504	0.17527
v_{min} on $y = 0.5$	-0.252853	-0.253565	-0.253704	-0.24533
ψ bottom-right / left eddy	1.37×10^{-5} / 2.07×10^{-6}	1.35×10^{-5} / 1.94×10^{-6}	1.34×10^{-5} / 1.90×10^{-6}	1.25×10^{-5} / 1.75×10^{-6}
$\max\lvert u-u_{Ghia}\rvert$, $\max\lvert v-v_{Ghia}\rvert$	0.0049, 0.0097	0.0048, 0.0099	0.0048, 0.0100	–

Table 3: Benchmark quantities (extrema located to sub-grid accuracy by quadratic fits; extremum values from the tightly converged runs).

ψ_{min} and the vortex centre agree with the benchmark to 0.08% and within one grid cell of the benchmark grid. The centreline extrema differ from Ghia's tabulated values by 1.5–3.4%, and these differences do *not* shrink with refinement: Ghia's values are samples on a 129×129 grid at fixed points (e.g. v is tabulated at $x = 0.8047$, whereas the minimum lies at $x = 0.8105$), so the benchmark itself carries an error of this size. The grid study below shows that the present results are converged to much better than this.

4.4 Grid convergence study

Three grids with refinement ratios $r_{21} = 1.5625$ and $r_{32} = 2$ were run to the tight tolerance ($|\partial\omega/\partial t| < 10^{-4}$, so that the residual transient does not contaminate the differences). The observed order p, the Richardson-extrapolated value and the fine-grid convergence index (GCI, safety factor 1.25) follow Celik et al. (2008):

$$p = \frac{1}{\ln r_{21}}\left|\ln\left|\frac{\varepsilon_{32}}{\varepsilon_{21}}\right| + \ln\frac{r_{21}^{p}-s}{r_{32}^{p}-s}\right|, \quad \phi_{ext} = \frac{r_{21}^{p}\phi_1 - \phi_2}{r_{21}^{p}-1}, \quad \mathrm{GCI} = \frac{1.25\,|(\phi_1-\phi_2)/\phi_1|}{r_{21}^{p}-1}. \tag{7}$$

Quantity	observed order p	extrapolated value	GCI_{fine} [%]
ψ_{min}	2.02	-0.103522	0.022
u_{min}	2.01	-0.214046	0.036
v_{max}	2.02	0.179573	0.048
v_{min}	2.01	-0.253801	0.047

Table 4: Grid convergence (`results/grid_study.txt`).

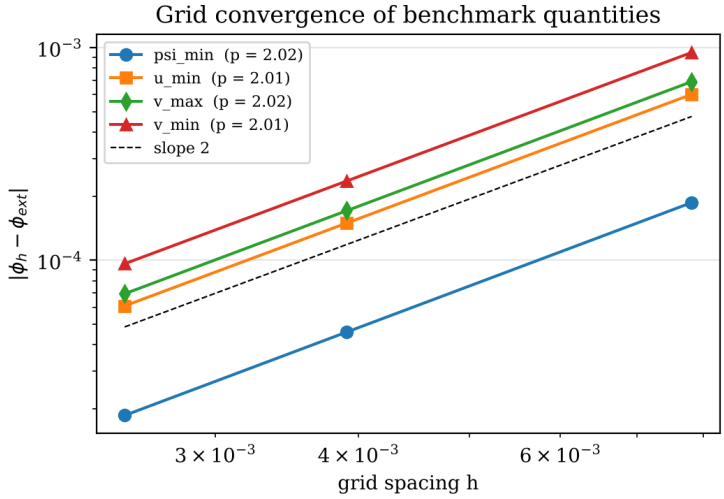


Figure 5: Distance from the extrapolated value against grid spacing: all quantities converge at second order.

The observed order matches the formal second order of the scheme, which also confirms that the Thom wall condition does not degrade the interior accuracy. The default stopping tolerance leaves a small transient error: on the 401^2 grid v_{max} is 0.18006 with the default test and 0.17950 with the tight one (0.3%); for benchmark-quality numbers the tight tolerance (`bash run.sh study`) should be used.

4.5 Run times

Grid	steps (default)	CPU emulation, 1 core [s]	per node and step [ns]
129^2	10 940	3.4	19
257^2	39 616	37.5	14
401^2	95 127	710.6	46

Table 5: Serial CPU baseline (emulated kernels, `g++ -O3`). The GPU wall times are written by `run.sh` to `results/timing.txt` when the code is run on a machine with `nvcc` and an NVIDIA GPU.

The step count grows like h^{-2} at constant d and the work per step like h^{-2}, so the total cost grows like h^{-4}; this is the regime where the per-node parallelism of the GPU pays off, provided the per-step host reduction mentioned above is moved to the device.

5 Conclusions

- The CUDA vorticity–stream function solver, after correcting an unstable time step (6.4 times the diffusion limit that the original notes themselves derive), transposed array indexing in the output and velocity routines, a per-step output that would have written ∼550 GB, and a stopping test that ignored the time step, reproduces the Re = 100 cavity flow.
- $\psi_{min} = -0.10350$ at $(0.616, 0.737)$ agrees with Ghia et al. to 0.08%; both corner eddies are captured.
- A three-grid study shows second-order convergence ($p = 2.01$–2.02) with GCI $< 0.05\%$; the extrapolated benchmark values are $\psi_{min} = -0.10352$, $u_{min} = -0.21405$, $v_{max} = 0.17957$, $v_{min} = -0.25380$.
- All kernels were verified by serial emulation; GPU timings are produced by `run.sh` on a CUDA-capable machine.

References

U. Ghia, K.N. Ghia, C.T. Shin, High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method, *J. Comput. Phys.* 48 (1982) 387–411.
I.B. Celik, U. Ghia, P.J. Roache, C.J. Freitas, H. Coleman, P.E. Raad, Procedure for estimation and reporting of uncertainty due to discretization in CFD applications, *J. Fluids Eng.* 130 (2008) 078001.
A. Thom, The flow past circular cylinders at low speeds, *Proc. R. Soc. Lond. A* 141 (1933) 651–669.
H. Juujärvi, I. Kinnunen, Lid driven cavity flow using stencil-based numerical methods (reference used in the original notes).
NVIDIA, *CUDA C++ Programming Guide* (unified memory, thread hierarchy).

Reproducing the results

`bash run.sh` (GPU if available, then CPU emulation of the 129^2, 257^2, 401^2 grids, about 13 min on one core); `bash run.sh study` adds the tightly converged grid study (about 10 more minutes); `bash run.sh gpu` runs only the GPU cases. Post-processing: `python3 code/postprocess.py`.

A Source code

```
// 2-D lid-driven cavity, vorticity-stream function formulation, CUDA (OE5450 Tutorial 6).
// Original author: Karthick J (OE23S018). Corrections marked [FIX], additions [ADDED].
#include <stdio.h>
#include <math.h>
#include <cuda.h>
#include <cuda_runtime.h>
#include <sys/stat.h>
#include <time.h> // [ADDED] wall-clock timing

// Simulation Parameters
#define L 1.0
#define DX 0.0025
#define DY 0.0025
#define Re 100.0
#define Dt 1.5e-4 // [FIX: was 0.001. Explicit vorticity diffusion needs Dt/(Re*DX^2) <= 1/4, i.e.
                  // Dt <= DX^2*Re/4 = 1.5625e-4 (the bound derived in the report); 0.001 gives 1.6 -> blow-up]
#define MaxIter 400000 // [FIX: was 50000 (t = 7.5 at the stable Dt): not enough to reach steady state]
#define Error (1.0e-2 * Dt) // [FIX: was 1e-5. The test is on max|w^{n+1}-w^n| = Dt*|dw/dt|, so a fixed
                            // 1e-5 only meant |dw/dt| < 0.01 at the original Dt = 1e-3. With the stable
                            // Dt = 1.5e-4 it stopped the 401x401 run at t = 8.7 while psi (one red-black
                            // sweep per step) still lagged: max|lap(psi)+w| = 0.11, psi_min 1.8% off Ghia.
                            // Scaling with Dt keeps |dw/dt| < 0.01 on every grid (= 1e-5 at Dt = 1e-3).]
#define U_Wall 1.0
#define NX ((int)(L/DX) + 1)
#define NY ((int)(L/DY) + 1)
#define OUTPUT_INTERVAL 20000 // [FIX: was 1 -> one 11 MB file per iteration (~550 GB in total)]

// CUDA Error Checking
#define CHECK(call) { \
    cudaError_t err = call; \
    if(err != cudaSuccess) { \
        fprintf(stderr, "CUDA Error: %s:%d - %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(1); \
    } \
}

// Kernel Prototypes with C linkage
#ifdef __cplusplus
extern "C" {
#endif
__global__ void apply_boundary_conditions(double *Vort, double *Str);
__global__ void update_vorticity(double *Vort, double *Vop, double *Str);
__global__ void solve_stream_function(double *Str, double *Vort, int phase);
__global__ void calculate_velocity(double *u, double *v, double *Str);
#ifdef __cplusplus
}
#endif

// ParaView-compatible Tecplot output
void write_tecplot2D(double* u, double* v, double* vort, double* stream,
                     int nx, int ny, double dx, double dy, int iter) {
    char filename[256];
    FILE *fp;

    // Create output directory
    struct stat st = {0};
    if (stat("output", &st) == -1) mkdir("output", 0700);

    sprintf(filename, "output/flow_%04d.dat", iter);
    fp = fopen(filename, "w");

    // Write header
    fprintf(fp, "TITLE = \"Lid-Driven Cavity Flow\"\n");
```

```
    fprintf(fp, "VARIABLES = \"X\", \"Y\", \"U\", \"V\", \"Vorticity\", \"Stream\"\n");
    fprintf(fp, "ZONE I=%d, J=%d, DATAPACKING=POINT\n", nx, ny);

    // Write data with fixed-width format
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            double x = i * dx;
            double y = j * dy;
            int idx = i * ny + j; // [FIX: solver arrays are stored as [i*NY + j] (i along x);
                                  // j*nx + i wrote every field transposed]
            fprintf(fp, "%10.6e %10.6e %10.6e %10.6e %10.6e %10.6e\n",
                    x, y,
                    u[idx], v[idx],
                    vort[idx], stream[idx]);
        }
    }
    fclose(fp);
}

int main() {
    // Allocate Unified Memory
    double *Vort, *Vop, *Str, *u, *v;
    CHECK(cudaMallocManaged(&Vort, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&Vop, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&Str, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&u, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&v, NX*NY*sizeof(double)));

    // Initialize Arrays
    for(int i=0; i<NX*NY; i++) {
        Vort[i] = 0.0; Vop[i] = 0.0; Str[i] = 0.0; u[i] = 0.0; v[i] = 0.0;
    }

    // CUDA Grid Configuration
    dim3 block(16, 16);
    dim3 grid((NX + block.x - 1)/block.x, (NY + block.y - 1)/block.y);

    // Main Simulation Loop
    int converged = 0, last_iter = 0;
    struct timespec t_start, t_end; // [ADDED] timing of the solver loop
    clock_gettime(CLOCK_MONOTONIC, &t_start);
    FILE *hist = fopen("convergence_history.txt", "w"); // [ADDED]
    for(int iter=0; iter<MaxIter && !converged; iter++) {
        last_iter = iter;
        // Apply Boundary Conditions first
        apply_boundary_conditions<<<grid, block>>>(Vort, Str);
        CHECK(cudaDeviceSynchronize());

        // Store previous vorticity (with BCs applied)
        CHECK(cudaMemcpy(Vop, Vort, NX*NY*sizeof(double), cudaMemcpyDeviceToDevice));

        // Update Vorticity using previous state (Vop)
        update_vorticity<<<grid, block>>>(Vort, Vop, Str);
        CHECK(cudaDeviceSynchronize());

        // Solve Stream Function (Red-Black Gauss-Seidel)
        for(int phase=0; phase<2; phase++) {
            solve_stream_function<<<grid, block>>>(Str, Vort, phase);
            CHECK(cudaDeviceSynchronize());
        }

        // Check convergence every iteration after 10 steps
        if(iter > 1) {
            double max_error = 0.0;
            for(int idx=0; idx<NX*NY; idx++) {
                double error = fabs(Vort[idx] - Vop[idx]);
                if(error > max_error) max_error = error;
            }
            if(iter % 1000 == 0) { // [CHANGED: print/log every 1000 iterations instead of every one]
                printf("Iter %d: Max Error = %e\n", iter, max_error);
                fprintf(hist, "%d %.6e %.6e\n", iter, iter * Dt, max_error);
            }
            if(max_error < Error) {
                converged = 1;
                printf("Converged after %d iterations\n", iter);
```

```
            }
        }

        // Output every OUTPUT_INTERVAL steps
        if(iter % OUTPUT_INTERVAL == 0) {
            calculate_velocity<<<grid, block>>>(u, v, Str);
            CHECK(cudaDeviceSynchronize());
            write_tecplot2D(u, v, Vort, Str, NX, NY, DX, DY, iter);
        }
    }

    // Final Output
    calculate_velocity<<<grid, block>>>(u, v, Str);
    CHECK(cudaDeviceSynchronize());
    write_tecplot2D(u, v, Vort, Str, NX, NY, DX, DY, 999999); // [FIX: fixed name for the final field]
    clock_gettime(CLOCK_MONOTONIC, &t_end); // [ADDED]
    printf("Grid %d x %d, Dt = %g, steps = %d, wall time = %.3f s\n", NX, NY, Dt, last_iter + 1,
           (t_end.tv_sec - t_start.tv_sec) + 1e-9 * (t_end.tv_nsec - t_start.tv_nsec));

    fclose(hist);
    // Free Memory
    CHECK(cudaFree(Vort));
    CHECK(cudaFree(Vop));
    CHECK(cudaFree(Str));
    CHECK(cudaFree(u));
    CHECK(cudaFree(v));

    return 0;
}

// Boundary Conditions Kernel (equations 30-33 from PDF)
__global__ void apply_boundary_conditions(double *Vort, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i >= NX || j >= NY) return;

    const double h2 = DX * DX;

    // Top Boundary (equation 30)
    if(j == NY-1 && i > 0 && i < NX-1) {
        Vort[i*NY + j] = (-2*Str[i*NY + (j-1)]/h2) - (2*U_Wall/DX);
    }
    // Bottom Boundary (equation 31)
    else if(j == 0 && i > 0 && i < NX-1) {
        Vort[i*NY + j] = -2*Str[i*NY + (j+1)]/h2;
    }
    // Left Boundary (equation 32)
    else if(i == 0 && j > 0 && j < NY-1) {
        Vort[i*NY + j] = -2*Str[(i+1)*NY + j]/h2;
    }
    // Right Boundary (equation 33)
    else if(i == NX-1 && j > 0 && j < NY-1) {
        Vort[i*NY + j] = -2*Str[(i-1)*NY + j]/h2;
    }
}

// Vorticity Update Kernel (equation 16 from PDF)
__global__ void update_vorticity(double *Vort, double *Vop, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i <= 0 || i >= NX-1 || j <= 0 || j >= NY-1) return;

    const double inv2h = 1.0/(2.0*DX);
    const double h2 = DX*DX;

    // Calculate derivatives using previous state (Vop)
    double dStr_dy = (Str[i*NY + (j+1)] - Str[i*NY + (j-1)]) * inv2h;
    double dStr_dx = (Str[(i+1)*NY + j] - Str[(i-1)*NY + j]) * inv2h;
    double dVop_dx = (Vop[(i+1)*NY + j] - Vop[(i-1)*NY + j]) * inv2h;
    double dVop_dy = (Vop[i*NY + (j+1)] - Vop[i*NY + (j-1)]) * inv2h;

    // Calculate Laplacian (equation 16)
    double laplacian = (Vop[(i+1)*NY + j] + Vop[(i-1)*NY + j] +
```

```
                    Vop[i*NY + (j+1)] + Vop[i*NY + (j-1)] -
                    4*Vop[i*NY + j]) / h2;

    // Update vorticity (equation 16)
    Vort[i*NY + j] = Vop[i*NY + j] + Dt * (
        (laplacian/Re) - (dStr_dy*dVop_dx - dStr_dx*dVop_dy)
    );
}

// Stream Function Solver (equation 18 from PDF)
__global__ void solve_stream_function(double *Str, double *Vort, int phase) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if((i+j)%2 != phase) return;
    if(i <= 0 || i >= NX-1 || j <= 0 || j >= NY-1) return;

    Str[i*NY + j] = 0.25 * (
        Str[(i+1)*NY + j] + Str[(i-1)*NY + j] +
        Str[i*NY + (j+1)] + Str[i*NY + (j-1)] +
        DX*DX*Vort[i*NY + j]
    );
}

// Velocity Calculation (equations 34-35 from PDF)
__global__ void calculate_velocity(double *u, double *v, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i >= NX || j >= NY) return;

    // [FIX: wall values. The original returned early for all boundary nodes, so the lid
    // velocity line below was unreachable; walls are now set explicitly.]
    if(i == 0 || i == NX-1 || j == 0 || j == NY-1) {
        u[i*NY + j] = (j == NY-1 && i > 0 && i < NX-1) ? U_Wall : 0.0;
        v[i*NY + j] = 0.0;
        return;
    }

    const double inv2h = 1.0 / (2.0 * DX);

    // [FIX: index as [i*NY + j] like every other kernel; the original used [j*NX + i], which
    // differentiated psi along the wrong direction (fields transposed)]
    // u = dψ/dy (equation 34)
    u[i*NY + j] = (Str[i*NY + (j+1)] - Str[i*NY + (j-1)]) * inv2h;

    // v = -dψ/dx (equation 35)
    v[i*NY + j] = -(Str[(i+1)*NY + j] - Str[(i-1)*NY + j]) * inv2h;
}
```